LR-Agent / Agent 后端入门

第08章
后端测试调试与证据

从一个失败输入出发，验证协议、隔离故障并留下可复现实验

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 区分单元、合同、集成与真实任务验收的证明范围。
- 用离线假对象和失败注入定位事件、工具与恢复边界。
- 阅读本轮13项测试结果，分清历史证据与当前执行。
- 建立功能、模型质量、人工效率分开的项目复盘报告。

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

8.1 为什么能聊天不等于后端可靠 . . . 3
8.2 名词解析：单元、合同、集成和验收 . . . 4
8.3 运行前：固定解释器与输入 . . . 5
8.4 本轮实测：13项后端离线测试通过 . . . 6
8.5 假模型：固定输出以隔离框架问题 . . . 7
8.6 流式调试：先区分chunk和完整事件 . . . 8
8.7 完整实验：验证真实SSE编码边界 . . . 9
8.8 故障注入：主动制造一类可解释失败 . . . 10
8.9 完整实验：给读取工具注入超时 . . . 11
8.10 协议对照：比较逐帧行为而非一句回答 . . . 12
8.11 真实故障案例：日志断管与Agent错误要分层 . . . 13
8.12 调试与追踪：先本地最小证据，再按需扩展 . . . 14
8.13 三套评价：功能、检测质量、人工效率 . . . 15
8.14 完整实验：只学公式，不虚构项目提升 . . . 16
8.15 练习：从一个失败定位到最小回归 . . . 17
8.16 面试复盘：给出证据链而非测试数量 . . . 18
8.17 源码图解：从函数签名读到返回值 . . . 19

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

8.1 为什么能聊天不等于后端可靠

场景：Agent回答“已完成”，但执行方没有收到tool_pending；或者工具回传后再次执行，产生重复提案。仅阅读最后一句话很难发现问题。测试要观察明确的输入、事件序列、执行次数与副作用，再决定问题位于哪层。



图8-1 概念测试闭环：先定义什么叫正确

想验证的问题	适合看什么	不足以证明
字段是否稳定	序列化函数与合同断言	真实服务可达
工具是否重复	已完成ID与执行次数	任意并发可靠
真实任务能否闭环	真实依赖和回读验收	整体检测质量

本章范围

主修Python后端测试；操作系统日志断管仅作为故障定位案例，不要求学习桌面框架。测试是工程证据，不是把通过数量当作个人贡献。

8.2 名词解析：单元、合同、集成和验收

Unit test（单元测试）隔离一个小行为；Contract test（合同测试）验证边界双方约定的数据与时序；Integration test（集成测试）检查真实组件组合；Acceptance（验收）围绕一个业务任务判断是否满足要求。这些名称反映关注点，不保证覆盖率天然完整。

类别	项目中的例子	主要局限
单元	路径提取、字段格式化	依赖已被隔离
合同	pending/resume字段与顺序	常用固定模型和存根
集成	框架Agent+适配器+HTTP	仍可能替换外部服务
真实验收	真实模型与推理到重启回读	样本和环境范围有限

一个测试可以同时具有合同和集成属性。例如生产适配器配合脚本化模型经过HTTP/SSE层，既验证接口约定，也验证内部装配；但它依然没有测云模型的稳定性。读测试时先找哪些依赖是真的，哪些被替换。

pytest是什么

pytest是Python测试运行器。以test_开头的函数通常会被收集；assert表达必须成立的条件。断言失败意味着这个输入不满足约束，而不是证明整个项目毫无价值。

8.3 运行前：固定解释器与输入

本项目有Agent与inference等不同环境。运行测试首先确认解释器；不依赖IDE自动选环境，也不把YOLO依赖混装进Agent环境。本章统一从项目根目录设置导入路径并关闭可选追踪，随后进入local-agent目录执行pytest。

完整教学实验运行约定：本章可复制PowerShell运行命令

```powershell
$env:PYTHONPATH = 'D:\Study\LR-Agent\LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
Set-Location 'D:\Study\LR-Agent\LR-Agent\vendor\local-agent'
& 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe' `
  -B -m pytest -q -p no:cacheprovider tests/test_assist_vision.py
```

命令片段	含义	为什么保留
-m pytest	以当前Python运行pytest	避免调到其他环境
-q	简洁输出	突出失败或通过结果
-B / no:cacheprovider	不写pyc/pytest缓存	减少教学临时文件
PYTHONPATH	指定项目模块根目录	保证app包可导入

路径可替换

如果你把仓库放在别处，只替换根路径；不要复制个人凭据。后面的my_ch08实验从项目根目录运行，使用同一解释器与环境变量。

8.4 本轮实测：13项后端离线测试通过

本轮于2026-10-17使用正式Agent环境选跑13项测试，结果为13 passed in 2.88s。它们均为离线测试，没有请求云模型、启动YOLO或依赖页面操作。耗时只描述这次机器上的运行，不作为性能指标。

选测范围	测试内容	本次结果
路径/视觉决策	图片路径提取与5项触发规则	6项通过
旧路径合同	pending/resume与首轮前取消	2项通过
候选视觉路径	触发、跳过、轮次、失败、主动附图	5项通过

本轮后端离线验证 / 真实stdout阅读图

原始记录：chapter08_pytest.txt；命令换行排版，结果保持原样

```
LR-Agent backend learning verification | 2026-10-17
Working directory: LR-Agent/vendor/local-agent
Interpreter: .venvs/langchain-main/Scripts/python.exe
Tracing disabled. Offline fake models and tools only.
Command: python -B -m pytest -q -p no:cacheprovider tests/test_assist_vision.py
tests/test_vision_bootstrap.py tests/test_agent_migration_contract.py tests/test_langchain_runtime_sta
ge5a.py::test_vision_fallback_triggers_on_first_round_without_tool_call tests/test_langchain_runtime_s
tage5a.py::test_vision_fallback_skipped_when_first_round_has_real_tool_call tests/test_langchain_runti
me_stage5a.py::test_vision_fallback_round_guard_counts_second_round_unconditionally tests/test_langcha
in_runtime_stage5a.py::test_vision_fallback_read_failure_becomes_tool_result_not_error_frame tests/tes
t_langchain_runtime_stage5a.py::test_active_vision_tool_call_appends_multimodal_message_after_tool_res
ult

.............                                                            [100%]
13 passed in 2.88s

Scope: path parsing, vision decisions, pending/resume contract, cancellation,
candidate vision fallback guards/read failures/active image attachment.
No cloud model, YOLO inference, UI, or complete regression suite executed here.
```

图8-2 本轮后端测试终端证据：以实际保存的13项pytest运行记录截图为准；属于离线验证。

源码定位：tests/test_assist_vision.py、test_vision_bootstrap.py、test_agent_migration_contract.py、test_langchain_runtime_stage5a.py。旧合同文件测试stream_assist；候选视觉测试驱动官方Agent路径，二者的被测范围需要分别解释。

历史结果单独标注

568通过/2跳过和16场景逐帧对照属于已有验收记录；本页没有声称重新执行整套测试。通过少量目标测试也不能替代未来改动后的完整回归。

8.5 假模型：固定输出以隔离框架问题

Fake（假对象）是测试依赖的可控替身。真实模型可能因采样、服务状态或配置改变输出；要检查后端如何处理tool_calls，应先让假模型固定提出同一调用，再断言结果。不使用网络并不等于伪造证据，前提是明确说明替换了什么。



图8-3 概念隔离：先验证确定性，再验证真实依赖

测试替身	适合验证	未覆盖
假factory	模型配置映射	模型执行能力
脚本化流式模型	chunk、tool call与调度	真实模型理解
固定工具函数	错误和结果封装	真实磁盘或推理依赖
真实模型冒烟	当前端到端可用	稳定指标与压力边界

要读完整定义

比较流协议时，假模型必须真正产生流式chunk。只实现_generate再比较工具结果，不能证明文本流和工具参数分片都被覆盖。完整项目假模型见experiments/compare_agent_contract.py::ScriptedStreamChat。

8.6 流式调试：先区分chunk和完整事件

Chunk（分片）是逐步到达的数据片段，未必是一条完整消息。工具参数可能分成多段JSON字符串；过早解析会报错或丢字段。后端适配器负责把框架事件转换成业务事件，再由SSE编码器输出；读日志时要知道当前观察的是哪一层。

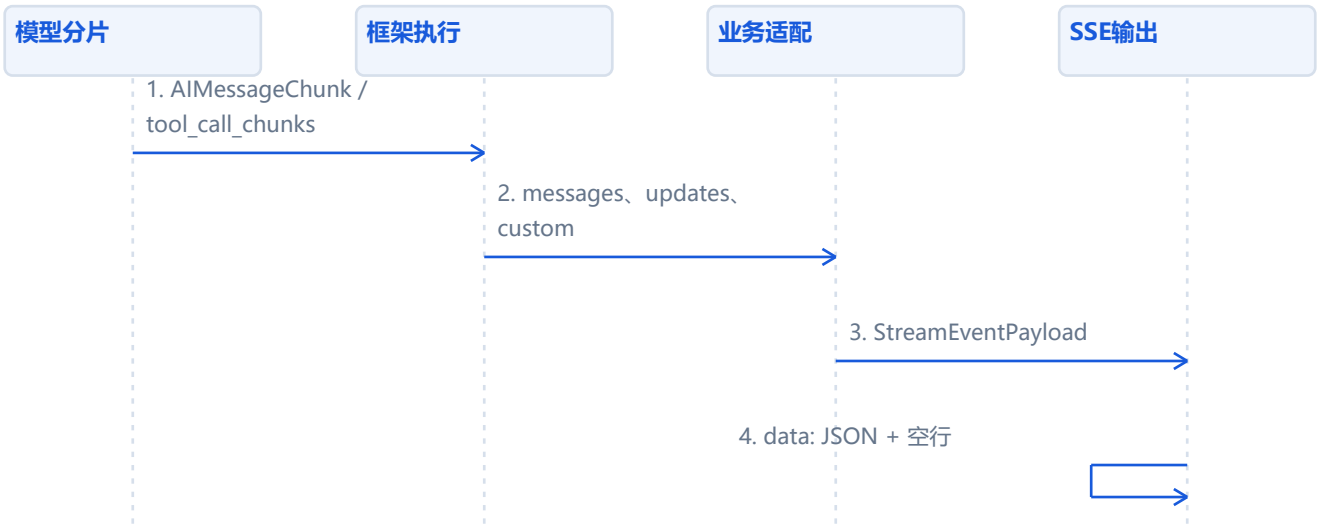


图8-4 实际分层关系：不要跨层比较原始形状

观察对象	适合断言	常见误区
模型chunk	文本/参数增量是否合并	每片都当完整JSON
业务事件	类型、call id和状态	用最终回答替代事件检查
SSE帧	字段名、分隔符与顺序	把网络包当事件边界

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::sse_frame（等价换行显示）

```python
def sse_frame(payload: StreamEventPayload) -> str:
    return "data: " + json.dumps(
        payload.to_sse_dict(), ensure_ascii=False
    ) + "\n\n"
```

项目源码stream_agent_events使用原始流驱动和适配逻辑；to_sse_dict集中处理字段重命名。部分文件顶部保留迁移阶段说明，当前行为要结合函数体和测试核对，不从历史注释直接下结论。

8.7 完整实验：验证真实SSE编码边界

先从最窄的边界开始：一个内部事件怎样变成外部JSON键名？本实验调用生产sse_frame和Schema，确认同一tool_call_id、可选字段与空行分隔。它适合作为合同测试的起点。

完整教学实验 ch08_sse。在 study 自行新建 my_ch08_sse.py，复制全部代码。

```python
import json
from app.agent.langchain_runtime.entry import sse_frame
from app.schemas.agent import StreamEventPayload

event = StreamEventPayload(
    type='tool_result', tool_call_id='case-8',
    name='read_workspace_file', result='read complete',
)
frame = sse_frame(event)
assert frame.startswith('data: ') and frame.endswith('\n\n')
payload = json.loads(frame[6:].strip())
assert payload['toolCallId'] == 'case-8'
assert 'tool_call_id' not in payload and 'content' not in payload
print('type:', payload['type'])
print('call_id:', payload['toolCallId'])
print('keys:', ','.join(sorted(payload)))
print('blank_line_terminated:', frame.endswith('\n\n'))
```

运行：& $agentPy ".\study\my_ch08_sse.py"（工作区根目录）

本轮实际输出：

```
type: tool_result
call_id: case-8
keys: name,result,toolCallId,type
blank_line_terminated: True
```

sse_frame 是项目真实函数，接受 StreamEventPayload 并返回字符串。本实验只编码一帧，没有开启HTTP服务，也没有重跑完整网络流。

frame[6:] 跳过 data: 和一个空格；strip 去掉外围换行；json.loads 恢复字典。检查键名比只看中文摘要更能定位协议变化。

None字段没有输出，外部toolCallId使用驼峰命名。流的两个换行是事件分隔符，不是随意排版。生产接收方仍需处理网络分块与重连。

继续扩展的方法

可将type替换为其他实际事件，逐项核对序列化规则。若要证明完整HTTP流，还需调用相应接口并读取整个事件序列，不能停留在单帧实验。

8.8 故障注入：主动制造一类可解释失败

Failure injection（故障注入）指主动让指定依赖以可控方式失败，再观察系统是否按约定响应。比如让读文件函数抛TimeoutError，检查是否变成ok=false，而不是一味等待偶然出现的错误。先明确失败发生在哪层，再决定恢复策略。

注入位置	预期反应	不该声称
工具函数超时	结构化失败结果	所有网络中断可恢复
参数或路径无效	校验拒绝且无副作用	模型永不生成错误参数
取消发生在执行前	不发起写入工具	执行后可以物理回滚
日志读端关闭	诊断输出失效不拖垮任务	检测算法变准确



图8-5 概念故障定位顺序

重试不是万能恢复

读操作和有副作用的操作风险不同；当前工具重试白名单为空，不能宣称全部工具自动重试。即使未来启用，也必须验证幂等和取消边界。

8.9 完整实验：给读取工具注入超时

下面定义一个完整的最小假模型和一个必定超时的读取函数。execute是教学简化封装，不是生产Agent调度器；它帮助你理解依赖注入、异常分类和结果结构。真实中间件行为应再查项目测试。

完整教学实验 ch08_failure。在 study 自行新建 my_ch08_failure.py，复制全部代码。

```python
from app.agent.tools.tool_result import build_tool_result, parse_tool_result

class FakeModel:
    def propose(self):
        return {'name': 'read_demo', 'args': {'path': 'demo.txt'}}

def failing_reader(path):
    raise TimeoutError('demo timeout')

def execute(call, reader):
    try:
        content = reader(**call['args'])
        return build_tool_result(ok=True, tool=call['name'],
                                 status='completed', summary=content)
    except TimeoutError:
        return build_tool_result(ok=False, tool=call['name'],
                                 status='failed', summary='read timeout')

result = parse_tool_result(execute(FakeModel().propose(), failing_reader))
assert result is not None and result['ok'] is False
assert result['status'] == 'failed'
assert result['file_written'] is False
print('ok:', result['ok'])
print('summary:', result['summary'])
print('file_written:', result['file_written'])
```

运行：& $agentPy ".\study\my_ch08_failure.py"（工作区根目录）

本轮实际输出：

```
ok: False
summary: read timeout
file_written: False
```

FakeModel 的完整定义已经展示：它固定提出一次读取请求，不继承 BaseChatModel，不运行 LangChain 图。这里只隔离测试“工具失败如何变成结构化结果”。

reader 是作为参数传入的函数；reader(**call["args"]) 把字典展开成具名参数。这就是依赖注入：测试换执行依赖，不必请求真实服务。

只捕获预期的 TimeoutError，其他编程错误继续失败。实验没有自动重试，没有吞掉所有异常，也没有证明网络取消或真实模型的恢复能力。

为什么不except Exception

把所有异常都变成“读取失败”会掩盖类型错误、字段缺失等程序缺陷。教学先捕获一个已知错误，失败范围更容易解释。生产是否捕获其他异常需要按层级设计。

8.10 协议对照：比较逐帧行为而非一句回答

迁移Agent内核时，“两个版本都答得差不多”不足以保持兼容。调用方可能依赖tool_start早于tool_pending、调用ID不变或done只出现一次。项目已有compare_agent_contract脚本，用流式脚本化模型对旧路径与候选适配器逐帧比较。



图8-6 实际合同对照脚本的核心方法

要比较的点	为何重要	阅读位置
文本与reasoning增量	避免静默丢失流	ScriptedStreamChat/to_ai_chunks
tool call参数分片	避免提前解析或漏调用	流式场景输入
pending/resume字段	维护外部执行协议	候选适配器与合同断言
错误/取消/收尾	避免悬挂或重复done	各场景完整帧列

证据边界

已有记录为16/16场景一致，本章未再次执行这个完整脚本。脚本中的工具和模型是离线替身；这证明确定场景的协议兼容，不证明真实模型在所有输入上的等价性。

8.11 真实故障案例：日志断管与Agent错误要分层

本次制作期间出现EPIPE（broken pipe）：日志写端仍在写，读端已经关闭。它属于进程I/O故障，不能看到Agent任务失败就立即归因模型。修复后的保护逻辑只容忍关闭管道相关错误，其他流异常仍然可见。

观察证据	证明什么	不证明什么
OS读端真实关闭	确实触发管道错误	所有进程故障均解决
无保护对照失败	故障条件可复现	Agent业务逻辑有误
保护后后续任务完成	日志失效未阻断后续任务	推理精度或吞吐改善
11项相关测试通过	本轮修复目标通过检查	整个工程所有功能通过

可定位LR-Agent/src/main/logging/stdioGuard.ts和stdioGuard.test.ts。这里只学习测试设计：建立真实失败基线、加入保护、验证后续任务仍完成。无需把桌面框架实现加入后端主线，也不把该修复当成本人的学习改造。

异常不要全吞

EPIPE与ERR_STREAM_DESTROYED属于特定关闭错误；其余异常需要保留可见性。健康检查、任务错误和日志错误分别观察，避免用“进程没崩”替代“任务成功”。

8.12 调试与追踪：先本地最小证据，再按需扩展

Observability（可观测性）指通过日志、事件和指标理解内部行为。LangSmith可以辅助观察调用链，但不是运行Agent的必选项；本项目没有追踪Key也应能启动。学习阶段先用固定输入与脱敏事件复现问题，再决定是否需要远程追踪。

证据	建议记录	不要直接公开
请求摘要	任务类型、样本数量、测试ID	API Key、登录Token
事件轨迹	type、call id、状态、耗时	完整私人文档和绝对路径
异常摘要	错误类别、阶段与可复现输入	未审查的原始trace
测试报告	命令、环境、结论和边界	个人会话文件

一个本地调试问题至少带上：在哪个版本、哪个解释器、什么输入、期待什么、实际什么、最先偏离在哪一帧。可选追踪若开启，先确认数据范围和用户配置；教材实验统一关闭追踪，避免把学习请求发送到外部。

函数定位

先从agent.py::local_chat_stream进入，再根据现象去entry、middleware、resume或sse_adapter。不要一次通读所有文件后仍不清楚哪层应该负责。

8.13 三套评价：功能、检测质量、人工效率

项目面试经常追问“好多少”。先说明评价对象，才能避免数字失真。协议字段正确可以由确定性断言验证；模型质量需要独立标注集；人工效率需要同类任务的计时和返工数据。三者的分母、判定规则和样本来源都不一样。

评价对象	可能指标	记录前提
功能/协议	场景通过率、重复执行数	明确场景和断言
检测质量	Precision、Recall、IoU	类别/匹配规则和独立数据
人工效率	总耗时、修正次数、失败率	任务难度与对照条件
服务行为	延迟分位数、错误率	真实负载与环境范围

Precision（精确率）=正确预测/(正确预测+错误预测)，衡量输出里有多少是对的。Recall（召回率）=正确预测/(正确预测+漏检)，衡量目标里找回多少。IoU为框的交并比，通常用于判定预测与真值是否匹配；阈值必须提前固定。

避免误用accuracy

目标检测常有多框、漏检和重复框，不能直接把分类准确率名称套过来。先定义匹配，再计算指标；置信分数也不是数据集精确率。

8.14 完整实验：只学公式，不虚构项目提升

使用两组人工指定的计数学习评价公式。TP为匹配正确的预测，FP为误报，FN为漏检。真实检测要先完成一对一框匹配；本例从计数开始，刻意不隐藏成一个自动评估系统。

完整教学实验 ch08_metrics。在 study 自行新建 my_ch08_metrics.py，复制全部代码。

```python
def precision_recall(tp, fp, fn):
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    return precision, recall

cases = [
    {'name': 'easy', 'tp': 2, 'fp': 0, 'fn': 0},
    {'name': 'hard', 'tp': 1, 'fp': 1, 'fn': 2},
]
tp = sum(case['tp'] for case in cases)
fp = sum(case['fp'] for case in cases)
fn = sum(case['fn'] for case in cases)
precision, recall = precision_recall(tp, fp, fn)
assert (tp, fp, fn) == (3, 1, 2)
assert precision == 0.75 and recall == 0.6
print('teaching_counts:', tp, fp, fn)
print(f'precision: {precision:.2f}')
print(f'recall: {recall:.2f}')
print('project_benchmark: not_measured')
```

运行：& $agentPy ".\study\my_ch08_metrics.py"（工作区根目录）

本轮实际输出：

```
teaching_counts: 3 1 2
precision: 0.75
recall: 0.60
project_benchmark: not_measured
```

本例故意使用人工指定的计数，不是本项目检测结果。sum 将各样本计数求和，再计算整体精确率与召回率；没有对每张图百分比直接取平均。

真实检测评估需先固定类别、IoU匹配阈值、重复框规则与数据划分，才能得到TP/FP/FN。IoU是两个框交集面积除以并集面积。

0.75和0.60仅用于核对公式，禁止写为项目指标。输出最后一行明确说明项目质量基准尚未测量。

为什么先求和

不同图片目标数不同，直接平均每张图百分比可能改变权重。这里按全部计数汇总，输出仍只是教学数值；真正报告必须说明聚合方法与数据规模。

8.15 练习：从一个失败定位到最小回归

练习A：tool_result有内容但调用方不显示，先查什么?

参考：先检查是否输出SSE空行分隔、外部键是否toolCallId、事件type是否正确，再检查适配器和接收层。不先猜模型不够聪明。复制SSE实验，故意改一个键名，观察断言指出的差异。

练习B：旧合同通过，能否证明候选路径全覆盖?

参考：不能。先看测试到底调用stream_assist还是官方Agent与候选适配器。用当前候选路径的相关测试补证据；历史16场景对照可作为参考，但未来改动仍需要重新执行适用检查。

练习C：怎样证明超时后没有重复写?

参考：分别记录调用ID、执行计数、提案和文件变化，注入重复结果或取消边界，再检查幂等约束。仅输出一条“超时”摘要不足以证明没有副作用。本章读取实验没有覆盖写工具。

你的复现报告	要写清	验收方式
环境与版本	解释器、提交/工作区状态	可复制命令
输入与预期	最小输入和具体断言	别人能得到同一结果
结论与局限	本轮执行与未测范围	不扩大通过含义

下一步改造候选

可以选择一个尚缺的边界合同测试作为本人改造，但先证明确有缺口；不要把教材实验文件直接当作生产修复或简历功能。

8.16 面试复盘：给出证据链而非测试数量

30秒概念答法

“我把Agent测试分成可控离线验证和真实依赖验收：假模型固定工具请求，检查调用ID、事件顺序、结果和副作用；真实任务再验证模型、推理与保存回读。测试通过证明明确场景的行为，不直接证明检测精度或用户效率。”

2分钟项目答法

选择一个本人复现的失败输入，说明预期、最先偏离的事件、被替换的依赖和真实保留的模块。展示断言与运行结果，再解释它和真实任务验收的区别。若做过改造，提供本人提交、修改前失败与修改后通过；若只是阅读，就明确说阅读。

追问	建议结构	证据落点
为何使用假模型	先隔离非确定性，再测真实服务	完整替身定义和断言
怎么发现协议破坏	逐帧比字段和顺序	对照脚本首个差异
数字提升从哪来	数据、规则、对照和运行范围	原始计数与报告
线上失败如何排查	分层复现并保留可解释错误	脱敏事件与最小用例

个人证据清单

本章可记录3个实验与目标pytest的本人重跑结果。已有568项通过、历史16场景或本轮EPIPE修复只能按实际参与程度说明，不能直接写成本人新增或独立完成。

8.17 源码图解：从函数签名读到返回值

先看图中的真实路径与原始行号，再打开源文件跟读。签名说明输入条件，函数体说明处理，return说明交给下一层的数据。图为源码阅读辅助，不替代完整上下文。

真实源码阅读图 / sse_frame

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py

```
81    def sse_frame(payload: StreamEventPayload) -> str:
...   # 文档说明省略，以下保留原始行号
83        return "data: " + json.dumps(payload.to_sse_dict(), ensure_ascii=False) + "\n\n"
```

真实源码阅读图：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::sse_frame。按当前文件原始行号截取并排版生成。

先看什么	怎样读	自己记录什么
输入	区分实参、配置与上下文	哪些字段可缺失
处理	找条件、调用和错误分支	哪一步改变消息或执行
输出	确认返回值与接收者	怎样用实验检查

源码与证据索引

LR-Agent/vendor/local-agent/app/schemas/agent.py::StreamEventPayload.to_sse_dict

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::sse_frame

LR-Agent/vendor/local-agent/app/agent/tools/tool_result.py::build_tool_result/parse_tool_result

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/sse_adapter.py::stream_agent_events

LR-Agent/vendor/local-agent/tests/test_assist_vision.py::test_vision_relative_from_user_text

LR-Agent/vendor/local-agent/tests/test_vision_bootstrap.py::VisionAutoLoader分支测试

LR-Agent/vendor/local-agent/tests/test_agent_migration_contract.py::test_pending_resume_preserves_backend_sse_contract

LR-Agent/vendor/local-agent/tests/test_langchain_runtime_stage5a.py::视觉中间件测试

LR-Agent/vendor/local-agent/experiments/compare_agent_contract.py::ScriptedStreamChat/main

LR-Agent/src/main/logging/stdioGuard.ts::installStdioGuards

LR-Agent/src/main/logging/stdioGuard.test.ts::real OS pipe closure

docs/2026-10-17本地主流程验收记录.md::自动化回归与真实验收

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。